

Université Lyon 2, Lyon 3, ENSIB, ENS Lyon

Origine, circularité et transformation des motifs naturalistes

Mémoire présenté par Morgan RICHARD

Année universitaire 2025–2026

Remerciements

Je tiens à remercier...

Résumé

Ce mémoire porte sur...

Table des matières

Remerciements	i
Résumé	ii
Introduction	1
1 Topic modeling sur les romans d'Émile Zola	2
1.1 Présentation du corpus	2
2 Nettoyage des romans et insertion de ces derniers dans la base de données	3
2.1 Reflexion sur le stockage des données	3
2.2 Nettoyage des textes	3
2.3 Création de la base de données	4
2.4 création des informations pour la base de données	7

Introduction

Chapitre 1

Topic modeling sur les romans d'Émile Zola

1.1 Présentation du corpus

Chapitre 2

Nettoyage des romans et insertion de ces derniers dans la basse de données

2.1 Reflexion sur le stockage des données

Après avoir récupéré un nombre conséquent de romans naturalistes et non naturaliste, il a fallu réfléchir à un moyen de les stocker quelque part. Pour cela, j'ai réfléchi à plusieurs idées pour procéder. Le choix c'est porté sur une base de données SQL avec comme interface PHP admin. Cette base de données à d'abord été remplie en local pour effectuer des tests et ensuite, une fois que tout fonctionnait correctement, j'ai pu extraire un fichier *Dumps* et l'importer dans PHP Admin.

2.2 Nettoyage des textes

Après avoir réfléchi à la manière de stocker les romans, il a fallu réfléchir à un moyen de les nettoyer. En effet, les romans contiennent beaucoup de caractères spéciaux qui ne sont pas utiles pour l'analyse, notamment les caractères de mise en forme, la pagination, les notes de bas de page, les chapitres ainsi que les prologues. En effet, les prologues sont souvent des textes qui ne sont pas écrits par l'auteur du roman et qui ne sont donc pas représentatifs de son style d'écriture.

Les romans naturalistes et non naturalistes sont issus de types de données différents. Il y a trois groupes de formats de données : les romans au format *.epub*, les romans au format *.pdf* et les romans au format *.txt*¹.

Pour ce qui est des textes au format *.epub*, j'ai utilisé le logiciel *Calibre*² pour les

1. Ici, les textes au format *.txt* sont organisés en dossiers, c'est-à-dire qu'un fichier *.txt* correspond à une page du roman. Il a donc fallu les regrouper pour obtenir un seul fichier *.txt* par roman.

2. Lien du logiciel : <https://calibre-ebook.com/fr/about>, site consulté le 18 juin 2026.

convertir au format *.txt*. Pour les textes au format *.pdf*, j'ai utilisé le logiciel *Adobe Acrobat Pro* afin de les convertir au format *.txt*. Pour les textes au format *.txt*, j'ai utilisé un script Python³ pour regrouper les fichiers en un seul fichier par roman.

La grande majorité de mes romans étaient au format *.epub*. Après leur conversion au format *.txt*, j'ai donc utilisé un script Python⁴ pour nettoyer les textes. Ce script supprime les caractères spéciaux, les notes de bas de page, les chapitres et les prologues. Il supprime également les lignes vides et les espaces en trop. Le script est basé sur des expressions régulières pour identifier les éléments à supprimer.

Pour ce qui est du choix des éléments à supprimer, j'ai fait le choix de supprimer les chapitres, car ils ne sont pas représentatifs du style d'écriture de l'auteur. De plus, comme nous l'avons vu dans le chapitre 2, j'ai passé un détecteur de motifs sur les textes afin d'identifier les motifs récurrents. J'ai donc gardé uniquement le corps du texte pour l'analyse. Ce choix se justifie également par la volonté d'obtenir une normalisation entre les textes⁵(une ligne est égal a une phrase), ce choix peut etre mit en opposition avec le choix de garder la distinction entre les différents chapitres du romans mais je me defend en prenant en compte que certains romans ne sont pas découpés en chapitres. Les differents romans sont soient découpés en partie (exemple : Partie I, Partie II, etc.), soit en chapitres (exemple : Chapitre I, Chapitre II, etc.), soit avec une combinaison des deux (exemple : Partie I, Chapitre I, Chapitre II, Partie II, Chapitre III, etc.). Ou pour finir, le titre du chapitre est écrit en majuscules, avec des chiffres romains, ou sans chiffres romains. Il aurait donc fallu faire des ajustements à chaque fois pour identifier les chapitres. De plus, certains romans résultaient de l'assemblage de plusieurs fichiers *.txt*. Dans ces cas, la séparation entre les chapitres, initialement présente dans la structure HTML du roman, n'était plus conservée.

2.3 Création de la base de données

Après avoir nettoyé les textes, il a fallu créer une base de données pour stocker les informations sur les romans. J'ai choisi d'utiliser le système de gestion de base de données relationnelle *MySQL* pour créer la base de données.

Je vais présenter la structure de la base de données ainsi que les différentes tables qui la composent. La base de données est composée de plusieurs tables, chacune ayant un rôle spécifique dans le stockage des informations sur les romans.

Je vais choisir de les présenter dans le corps de texte de mon mémoire⁶ et non

3. Le script est disponible en annexe.

4. Le script est disponible en annexe. Il faut préciser que je n'ai pas utilisé un seul script, mais plusieurs. Il s'agira donc d'un script général. Il y a eu énormément de modifications internes, notamment pour les chapitres qui sont soit en majuscules, avec chiffres romains, ou sans chiffres romains. Il a donc fallu faire des ajustements à chaque fois.

5. certains textes de disposant pas de

6. Je précise aussi que tout les codes utilisé pour la présentation de ce mémoire sont disponible sur

en annexe pour éviter de surcharger l'annexe avec des informations qui ne sont pas directement liées à l'analyse des textes.

```
1 CREATE DATABASE romans_nlp;
2 USE romans_nlp;
3
4 -- Creation des tables pour la base de donnees des romans et
  des auteurs
5 CREATE TABLE auteurs (
6     id_auteur INT PRIMARY KEY AUTO_INCREMENT,
7     nom VARCHAR(100) NOT NULL,
8     prenom VARCHAR(100)
9 );
10
11 -- Table pour stocker les romans, avec leur titre et leur annee
  de publication
12 CREATE TABLE romans (
13     id_roman INT PRIMARY KEY AUTO_INCREMENT,
14     titre VARCHAR(255) NOT NULL,
15     annee_publication INT,
16     texte_brut LONGTEXT
17 );
18
19 -- Table d'association pour gerer la relation plusieurs-a-
  plusieurs entre auteurs et romans
20 CREATE TABLE auteurs_romans (
21     id_auteur INT NOT NULL,
22     id_roman INT NOT NULL,
23     PRIMARY KEY (id_auteur, id_roman),
24     FOREIGN KEY (id_auteur) REFERENCES auteurs(id_auteur),
25     FOREIGN KEY (id_roman) REFERENCES romans(id_roman)
26 );
27
28 -- Table pour stocker les segments extraits des romans, avec
  leur ordre de segmentation
29 CREATE TABLE segments (
30     id_segment INT PRIMARY KEY AUTO_INCREMENT,
31     id_roman INT NOT NULL,
32     ordre_segment INT NOT NULL,
33     texte_segment LONGTEXT NOT NULL,
34     FOREIGN KEY (id_roman) REFERENCES romans(id_roman)
```

```

35 );
36
37 /*
38 Après réflexion, il est également possible d'ajouter les dates
39 de naissance et de mort des auteurs afin d'enrichir la base de
40 données.
41 */
42 ALTER TABLE auteurs
43 ADD date_naissance INT;
44
45 ALTER TABLE auteurs
46 ADD date_mort INT;
47
48 -- Table pour stocker les phrases extraites des romans, avec
49 leur motif d'extraction
50 CREATE TABLE phrase_roman (
51     id_phrase INT PRIMARY KEY AUTO_INCREMENT,
52     id_roman INT NOT NULL,
53     ordre_phrase INT NOT NULL,
54     texte_phrase TEXT NOT NULL,
55     motif_phrase TEXT NOT NULL,
56     FOREIGN KEY (id_roman) REFERENCES romans(id_roman)
57 );

```

Listing 2.1 – Création de la base de données `romans_nlp`

Ce code permet de créer la base de données utilisée pour stocker les romans du corpus ainsi que les informations nécessaires à leur analyse. La base, nommée `romans_nlp`, est d'abord créée puis sélectionnée afin que toutes les tables suivantes y soient enregistrées.

La première table, `auteurs`, sert à stocker les informations relatives aux auteurs. Chaque auteur reçoit un identifiant unique, auquel sont associés son nom et, lorsque l'information est disponible, son prénom. Cette table est ensuite complétée par deux colonnes supplémentaires, `date_naissance` et `date_mort`, afin d'enrichir la description des auteurs et de permettre d'éventuelles analyses chronologiques.

La table `romans` contient les informations principales sur les œuvres du corpus. Elle associe à chaque roman un identifiant unique, un titre, une année de publication ainsi que le texte brut du roman. Le choix du type `LONGTEXT` pour le champ `texte_brut` permet de stocker des textes longs, ce qui est nécessaire dans le cas de romans complets.

La table `auteurs_romans` permet de faire le lien entre les auteurs et les romans. Elle sert à gérer une relation de type plusieurs-à-plusieurs : un auteur peut être associé à plusieurs romans, et un roman peut éventuellement être associé à plusieurs auteurs. Les clés étrangères `id_auteur` et `id_roman` permettent de relier cette table aux tables

auteurs et romans.

La table **segments** sert à stocker les segments extraits des romans. Chaque segment est rattaché à un roman grâce à la clé étrangère **id_roman**. Le champ **ordre_segment** permet de conserver l'ordre d'apparition des segments dans le texte d'origine. Cela est important, car même si le texte est découpé pour les traitements, il reste possible de retrouver sa progression initiale.

Enfin, la table **phrase_roman** permet de stocker les phrases extraites des romans. Chaque phrase est associée au roman dont elle provient, à son ordre d'apparition, à son contenu textuel et au motif qui a permis son extraction. Ce dernier champ est important, car il permet de garder une trace de la méthode utilisée pour sélectionner les phrases et de relier chaque extraction à un critère précis.

L'ensemble de cette structure permet donc de conserver à la fois les textes complets, leurs découpages en segments, les phrases extraites et les informations bibliographiques liées aux auteurs et aux œuvres. Elle constitue ainsi une base organisée pour les traitements de fouille de texte et d'analyse automatique du corpus.

2.4 création des informations pour la base de données

Après avoir créé les tables nécessaires pour stocker les informations sur les auteurs, les romans et les relations entre eux, il est temps d'insérer les données relatives aux frères Goncourt et à leurs œuvres.

```
1  /*
2  Insertion des freres Goncourt.
3  Il est aussi possible d'ajouter leurs dates de naissance et de
   mort
4  afin d'enrichir la base de donnees.
5  */
6  INSERT INTO auteurs (nom, prenom)
7  VALUES
8      ('de Goncourt', 'Edmond'),
9      ('de Goncourt', 'Jules');
10 /*
11 Insertion des romans des freres Goncourt
12 avec leur annee de publication.
13 */
14 INSERT INTO romans (titre, annee_publication)
15 VALUES
16     ('Charles Demailly', 1860),
```

```

17      ('Soeur Philomene', 1861),
18      ('Renee Mauperin', 1864),
19      ('Germinie Lacerteux', 1865),
20      ('Manette Salomon', 1867),
21      ('Madame Gervaisais', 1869),
22      ('La Fille Elisa', 1877),
23      ('Freres Zemganno', 1879),
24      ('La Faustin', 1882),
25      ('Cherie', 1884);
26  /*
27  Insertion des relations entre les auteurs et les romans.
28  Les six premiers romans sont associes aux deux freres Goncourt.
29  Les romans suivants sont associes uniquement a Edmond de
      Goncourt.
30  */
31  INSERT INTO auteurs_romans (id_auteur, id_roman)
32  VALUES
33      (1,1), (2,1),
34      (1,2), (2,2),
35      (1,3), (2,3),
36      (1,4), (2,4),
37      (1,5), (2,5),
38      (1,6), (2,6),
39      (1,7),
40      (1,8),
41      (1,9),
42      (1,10);
43  -- Mise a jour des dates de naissance et de mort de Jules de
      Goncourt.
44  UPDATE auteurs
45  SET date_naissance = 1830,
46      date_mort = 1870
47  WHERE id_auteur = 2;
48
49  -- Mise a jour des dates de naissance et de mort d'Edmond de
      Goncourt.
50  UPDATE auteurs
51  SET date_naissance = 1822,
52      date_mort = 1896
53  WHERE id_auteur = 1;

```

Listing 2.2 – Insertion des frères Goncourt dans la base de données

Ce code permet d'insérer dans la base de données les informations relatives aux frères Goncourt et à leurs romans. Dans un premier temps, les deux auteurs sont ajoutés dans la table **auteurs**. Ils reçoivent chacun un identifiant, qui permettra ensuite de les relier aux œuvres présentes dans la base.

La deuxième partie du code insère les romans des frères Goncourt dans la table **romans**. Chaque œuvre est associée à son titre et à son année de publication. Cette étape permet donc de constituer une partie du corpus à partir des romans retenus pour l'analyse.

Ensuite, la table **auteurs_romans** est utilisée pour associer les auteurs aux romans. Cette table est importante, car elle permet de gérer le cas particulier des œuvres écrites à deux mains. Les six premiers romans sont associés à Edmond et Jules de Goncourt, tandis que les romans publiés après la mort de Jules sont associés uniquement à Edmond. Cela permet de conserver une information plus précise sur l'attribution des œuvres.

Enfin, les deux dernières requêtes mettent à jour les dates de naissance et de mort des auteurs. Ces informations biographiques ne sont pas directement nécessaires pour stocker les textes, mais elles permettent d'enrichir la base de données. Elles peuvent aussi être utiles par la suite si l'on souhaite croiser les résultats de l'analyse avec des éléments chronologiques ou biographiques.

Ce passage sert donc à remplir une première partie de la base de données avec les auteurs, les romans et les relations entre eux. Il montre aussi l'intérêt de séparer les informations dans plusieurs tables : les auteurs sont stockés d'un côté, les romans de l'autre, et la relation entre les deux est conservée dans une table spécifique.

Il faut maintenant ajouter les romans dans la base de données, il peut y avoir plusieurs façons de le faire,

Annexe technique : code